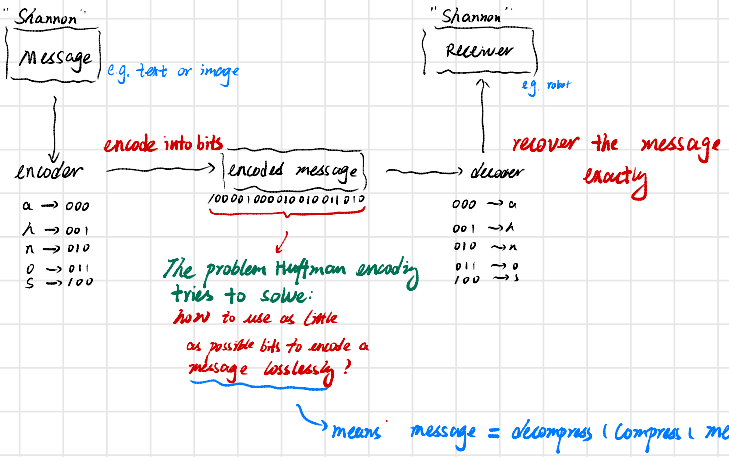
Huffman encoding is an algorithm for lossless data compression.

What is lossless data compression?

Suppose we want to send a message to a robot from our computer



↳ means: message = decompress(compress(message))

How to approach this problem?

If we have 2^n unique symbols, then a naive way is to give each symbol a equal length binary string.

e.g. if we have a, b, c, d

we can encode them as 00 01 10 11

However, if we know in the message we want to encode,

a: 50%
b: 25%
c: 12.5%
d: 12.5%

intuition: if a appears more often, then we may want to use less bit to represent it

Total length of encoded bits = $\sum_{x}^{n}$ → (number of symbols)

Expected length for a single token = $p(A)L(A) + p(B)L(B) + p(C)L(C) + p(D)L(D)$

↳ Denote as E, so we only need to minimize this

In our example

$E = 0.5 \times 2 + 0.25 \times 2 + 0.15 \times 2 + 0.15 \times 2 = 2$

Minimize expected length

Difficulty: use as less bits as possible
vs
we want lossless compression. So we can't use too little

e.g. if we now have

a	b	c	d
↓	↓	↓	↓
00	01	10	11

Cut to

a	b	c	d
↓	↓	↓	↓
0	01	10	11

↘ how to decode

010 11 → could be 0 | 10 | 11 → acd

↳ could be 01 | 0 | 11 → bad

The **key requirment** here:

if a Symbol (say s) takes a bit string $b_1 b_2 b_3 \cdots b_k$

then there should be no other Symbol use the whole $b_1 b_2 b_3 \cdots b_k$

e.g. $b_1 b_2 \cdots b_k 0$ is not allowed

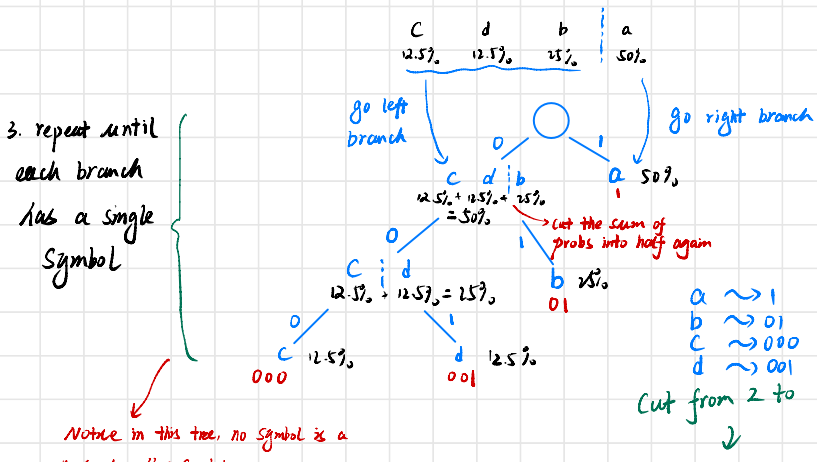
i.e. a Symbol cannot be a prefix for any other Symbol

Shannon Encoding or Shannon-Fano Coding

1. rank probability from low to high

C	d	b	a
12.5%	12.5%	25%	50%

2. divide the sum of probability into half.



3. repeat until each branch has a single Symbol

Notice in this tree, no Symbol is a prefix for other Symbols

To decode any bits string, simply walk down the tree.

e.g. 001 | 0 | 001

C a b d

a ~ 1
b ~ 01
c ~ 000
d ~ 001

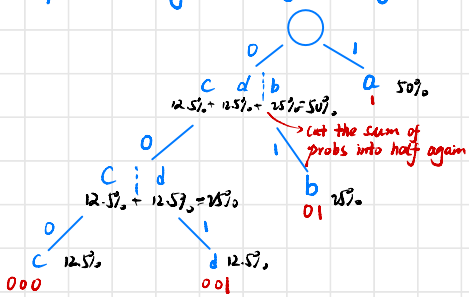
Cut from 2 to

$E = 1 \times 0.5 + 2 \times 0.25 + 3 \times 0.125 + 3 \times 0.125$

$= 1.75$

However, Shannon encoding is not optimal in general

In order to construct an optimal algorithm of encoding, let us take a look at the tree again:

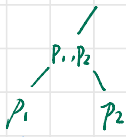


We have the following two important observations:

Observation 1: the number of bits for encoding each symbol equals to the depth of it in the tree.

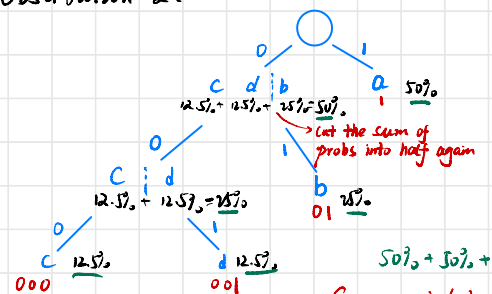
So for any two symbols a and b, if $p(a) < p(b)$ then in an optimal tree, we must have $depth(a) \geq depth(b)$

$\Rightarrow$ if $p_1 \leq p_2 \leq \cdots \leq p_K$, where $p_1 + \cdots + p_K = 1$



→ Symbols with lowest probabilities must be on the bottom layer of the tree

Observation 2:



$50\% + 50\% + 25\% + 25\% + 12.5\% + 12.5\% = 1.75$ → expected length

So expected length = Sum of prob at each node in the tree

↳ Why? for a symbol with depth k, it appears in k nodes.

According the above two observations, we have the expected length L for the optimal tree must satisfy that

$$L(p_1, p_2, \dots, p_k) = p_1 + p_2 + L((p_1+p_2), p_3, \dots, p_n), \text{ where } p_1 \le p_2 \le \dots \le p_k.$$

$L(p_1, p_2, \dots, p_k)$: expected length of the optimal tree

$L((p_1+p_2), p_3, \dots, p_n)$: expected length of the optimal tree when p_1 and p_2 is merged

Example

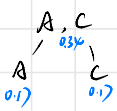
Symbols: D E A C B

probs: 0.15 0.16 0.17 0.17 0.35

Step 1: merge D and E

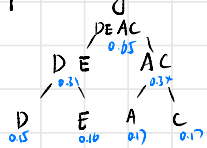
now we have A, C, DE, B (0.17, 0.17, 0.31, 0.35)

Step 2: merge A, C



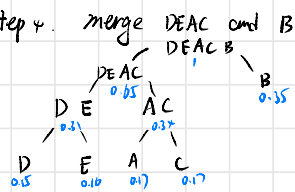
now we have DE, AC, B (0.31, 0.34, 0.35)

Step 3, merge DE and AC



now we have B (0.35), DEAC (0.65)

Step 4. merge DEAC and B



encode

So B ~ 1

D ~ 000

E ~ 001

A ~ 010

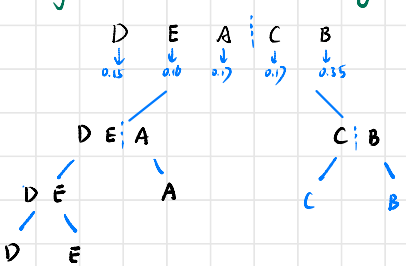
C ~ 011

$\rightsquigarrow L = 0.35 + 0.65 \times 3$

$= 0.35 + 1.95$

$= 2.3$

If use Shannon-Fano encoding:



better than

encode

So B → 11

D → 000

E → 001

A → 01

C → 10

$\rightsquigarrow L = 2\times0.35 + 3\times0.15 + 3\times0.16 + 2\times0.17 + 2\times0.17$

$= 0.7 + 0.45 + 0.48 + 0.68$

$= 2.31$